



UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

# Norme di Progetto<sub>G</sub>

Il nostro Way of Working<sub>G</sub>

|                             |                                                   |
|-----------------------------|---------------------------------------------------|
| <b>Versione</b>             | 2.0.0                                             |
| <b>Stato</b>                | Approvato                                         |
| <b>Redazione</b>            | Giulia Barzon, Joseph Grant,<br>Elisa Marchioro   |
| <b>Verifica<sub>G</sub></b> | Elisa Marchioro, Matteo Cuogo,<br>Chiara Grossele |
| <b>Approvazione</b>         | Matteo Cuogo                                      |
| <b>Proprietario</b>         | ByteMe                                            |
| <b>Uso</b>                  | Esterno                                           |
| <b>Destinatari</b>          | Prof. Tullio Vardanega<br>Prof. Riccardo Cardin   |

## Registro dei Cambiamenti

| Versione | Data       | Autore          | Descrizione                                                                                                | Verifica <sub>G</sub>           |
|----------|------------|-----------------|------------------------------------------------------------------------------------------------------------|---------------------------------|
| 2.0.0    | 24/04/2026 | Matteo Cuogo    | Approvazione documento                                                                                     |                                 |
| 1.1.0    | 20/04/2026 | Giulia Barzon   | Aggiunte sezioni 9, 10, 11                                                                                 | Matteo Cuogo                    |
| 1.0.8    | 10/04/2026 | Giulia Barzon   | Sezioni setup test in locale e Git <sub>G</sub> Flow                                                       | Matteo Cuogo                    |
| 1.0.7    | 26/03/2026 | Elisa Marchioro | Miglioramento sezione 5 e 6                                                                                | Chiara Grossele, Giulia Barzon  |
| 1.0.6    | 26/03/2026 | Elisa Marchioro | Fix sezione 5                                                                                              | Chiara Grossele                 |
| 1.0.5    | 25/03/2026 | Elisa Marchioro | Fix sezione 3 e aggiunta subsection "Metriche nel cruscotto di qualità <sub>G</sub> "                      | Matteo Cuogo, Tommaso Tom-bacco |
| 1.0.4    | 25/03/2026 | Elisa Marchioro | Miglioramento sezione 3                                                                                    | Matteo Cuogo, Tommaso Tom-bacco |
| 1.0.3    | 24/03/2026 | Giulia Barzon   | Miglioramento sezione 2                                                                                    | Matteo Cuogo, Tommaso Tom-bacco |
| 1.0.2    | 21/03/2026 | Elisa Marchioro | Mini fix                                                                                                   | Matteo Cuogo, Tommaso Tom-bacco |
| 1.0.1    | 16/03/2026 | Giulia Barzon   | Aggiunta modifica gestione sprint <sub>G</sub>                                                             | Matteo Cuogo, Tommaso Tom-bacco |
| 1.0.0    | 20/02/2026 | Matteo Cuogo    | Approvazione documento                                                                                     |                                 |
| 0.2.2    | 15/01/2026 | Giulia Barzon   | Aggiunta sezione su test automatici per qualità documentazione e sezione PoC                               | Matteo Cuogo                    |
| 0.2.1    | 11/01/2026 | Giulia Barzon   | Aggiunta sezione sul Piano di Qualifica                                                                    | Matteo Cuogo                    |
| 0.2.0    | 30/12/2025 | Elisa Marchioro | Aggiunte subsection sui requisiti in "Analisi dei requisiti <sub>G</sub> "                                 | Matteo Cuogo                    |
| 0.1.8    | 27/12/2025 | Joseph Grant    | Aggiunta sezione 'verifica <sub>G</sub> ' al versionamento <sub>G</sub>                                    | Elisa Marchioro                 |
| 0.1.7    | 16/12/2025 | Giulia Barzon   | Gestione comunicazione durante i periodi di vacanza                                                        | Elisa Marchioro                 |
| 0.1.6    | 5/12/2025  | Elisa Marchioro | Sistemata l'introduzione                                                                                   | Matteo Cuogo                    |
| 0.1.5    | 5/12/2025  | Elisa Marchioro | Aggiunta sezione documenti, modifica sezione pianificazione del lavoro, aggiunta sezione piano di progetto | Matteo Cuogo                    |
| 0.1.4    | 5/12/2025  | Elisa Marchioro | Aggiunta analisi dei requisiti <sub>G</sub>                                                                | Matteo Cuogo                    |
| 0.1.3    | 19/11/2025 | Giulia Barzon   | Aggiornamento regole GitHub <sub>G</sub> issues e dashboard                                                | Elisa Marchioro                 |
| 0.1.2    | 16/11/2025 | Elisa Marchioro | Aggiunta descrizione dei ruoli                                                                             | Matteo Cuogo                    |
| 0.1.1    | 14/11/2025 | Giulia Barzon   | Modifica struttura documento e aggiunta organizzazione e comunicazione                                     | Elisa Marchioro                 |
| 0.1.0    | 5/11/2025  | Elisa Marchioro | Stesura iniziale documento                                                                                 | Matteo Cuogo                    |

Tabella 1: Registro delle versioni del documento

# Indice

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                                 | <b>5</b>  |
| 1.1      | Scopo del documento . . . . .                                       | 5         |
| 1.2      | Cosa sono per noi le Norme di Progetto <sub>G</sub> ? . . . . .     | 5         |
| <b>2</b> | <b>Processi organizzativi</b>                                       | <b>6</b>  |
| 2.1      | Processo di gestione . . . . .                                      | 6         |
| 2.1.1    | Attività: organizzazione dei ruoli . . . . .                        | 6         |
| 2.1.2    | Attività: pianificazione degli sprint <sub>G</sub> . . . . .        | 6         |
| 2.2      | Processo di comunicazione . . . . .                                 | 7         |
| 2.2.1    | Attività: comunicazione interna . . . . .                           | 7         |
| 2.2.2    | Attività: comunicazione esterna . . . . .                           | 7         |
| 2.2.3    | Attività: comunicazione nei periodi di vacanza/sessione . . . . .   | 7         |
| 2.3      | Processo di Formazione . . . . .                                    | 8         |
| 2.3.1    | Attività: autoformazione individuale . . . . .                      | 8         |
| <b>3</b> | <b>Gestione del codice e documentazione</b>                         | <b>9</b>  |
| 3.1      | Processo di gestione della documentazione . . . . .                 | 9         |
| 3.1.1    | Attività: organizzazione della documentazione . . . . .             | 9         |
| 3.1.2    | Attività: sviluppo della documentazione . . . . .                   | 10        |
| 3.1.3    | Attività: versionamento <sub>G</sub> della documentazione . . . . . | 11        |
| 3.1.4    | Attività: verifica <sub>G</sub> della documentazione . . . . .      | 11        |
| 3.1.5    | Sito web della documentazione . . . . .                             | 12        |
| 3.2      | Processo di gestione del codice . . . . .                           | 12        |
| 3.2.1    | Attività: gestione delle attività di sviluppo . . . . .             | 12        |
| 3.2.2    | Attività: metodologia Git <sub>G</sub> Flow . . . . .               | 13        |
| 3.2.3    | Attività: automazione dei test pre-push . . . . .                   | 13        |
| 3.2.4    | Attività: gestione delle dipendenze (requirements.txt) . . . . .    | 14        |
| <b>4</b> | <b>Documenti</b>                                                    | <b>15</b> |
| 4.1      | .github <sub>G</sub> /workflows . . . . .                           | 15        |
| 4.2      | Candidatura . . . . .                                               | 15        |
| 4.3      | RTB . . . . .                                                       | 15        |
| 4.4      | PB . . . . .                                                        | 15        |
| 4.5      | Documenti interni . . . . .                                         | 16        |
| 4.6      | Verbalì esterni . . . . .                                           | 16        |
| 4.7      | Verbalì interni . . . . .                                           | 16        |
| 4.8      | docs . . . . .                                                      | 16        |
| 4.9      | scripts . . . . .                                                   | 16        |
| 4.10     | src . . . . .                                                       | 16        |
| 4.11     | README.md . . . . .                                                 | 16        |
| <b>5</b> | <b>Analisi dei Requisiti<sub>G</sub> (AdR)</b>                      | <b>17</b> |
| 5.1      | Studio dei casi d'uso . . . . .                                     | 17        |
| 5.1.1    | Attività: identificazione iniziale dei casi d'uso . . . . .         | 17        |
| 5.1.2    | Attività: analisi individuale dei casi d'uso . . . . .              | 17        |
| 5.1.3    | Attività: revisione dei casi d'uso . . . . .                        | 18        |
| 5.1.4    | Attività: discussione dei casi d'uso con l'azienda . . . . .        | 18        |
| 5.2      | Studio dei requisiti . . . . .                                      | 18        |
| 5.2.1    | Attività: analisi individuale dei requisiti . . . . .               | 18        |
| 5.2.2    | Attività: revisione dei requisiti . . . . .                         | 19        |

|           |                                                                                                    |           |
|-----------|----------------------------------------------------------------------------------------------------|-----------|
| 5.3       | Scrittura del documento definitivo . . . . .                                                       | 19        |
| 5.4       | Giudizio sull'approccio adottato . . . . .                                                         | 19        |
| <b>6</b>  | <b>Piano di Progetto (PdP)</b>                                                                     | <b>20</b> |
| 6.1       | Attività: analisi dei rischi . . . . .                                                             | 20        |
| 6.2       | Metodo adottato . . . . .                                                                          | 20        |
| 6.3       | Attività: pianificazione e gestione degli sprint <sub>G</sub> . . . . .                            | 20        |
| 6.4       | Attività: consuntivazione degli sprint <sub>G</sub> . . . . .                                      | 21        |
| <b>7</b>  | <b>Piano di Qualifica (PdQ)</b>                                                                    | <b>22</b> |
| 7.1       | Scopo del documento . . . . .                                                                      | 22        |
| 7.2       | Contenuto e struttura . . . . .                                                                    | 22        |
| 7.3       | Metodologia di calcolo dell'Earned Value (EV) . . . . .                                            | 22        |
| 7.4       | Definition of Done (DoD) . . . . .                                                                 | 23        |
| 7.5       | Strumenti e metriche di qualità automatizzate . . . . .                                            | 23        |
| 7.5.1     | Qualità della documentazione . . . . .                                                             | 23        |
| 7.5.2     | Qualità del codice . . . . .                                                                       | 24        |
| 7.5.3     | Visualizzazione dei risultati . . . . .                                                            | 24        |
| 7.6       | Metriche nel cruscotto di qualità <sub>G</sub> . . . . .                                           | 24        |
| <b>8</b>  | <b>Sviluppo del POC<sub>G</sub></b>                                                                | <b>26</b> |
| 8.1       | Stack tecnologico . . . . .                                                                        | 26        |
| 8.2       | Struttura della repository <sub>G</sub> per il PoC . . . . .                                       | 26        |
| 8.3       | Norme per il frontend . . . . .                                                                    | 27        |
| 8.3.1     | Architettura . . . . .                                                                             | 27        |
| 8.3.2     | Convenzioni di nomenclatura . . . . .                                                              | 27        |
| 8.3.3     | Regole sui componenti . . . . .                                                                    | 27        |
| 8.3.4     | Regole sui custom hook . . . . .                                                                   | 28        |
| 8.3.5     | Gestione dello stato . . . . .                                                                     | 28        |
| 8.3.6     | Stile . . . . .                                                                                    | 28        |
| 8.4       | Norme per il backend . . . . .                                                                     | 28        |
| 8.4.1     | Architettura . . . . .                                                                             | 28        |
| 8.4.2     | Convenzioni di nomenclatura . . . . .                                                              | 29        |
| 8.4.3     | Gestione del database <sub>G</sub> . . . . .                                                       | 29        |
| 8.4.4     | Gestione delle sessioni e sicurezza . . . . .                                                      | 29        |
| 8.4.5     | Endpoint <sub>G</sub> REST . . . . .                                                               | 29        |
| 8.5       | Norme per Docker <sub>G</sub> e deployment . . . . .                                               | 30        |
| 8.5.1     | Struttura di docker <sub>G</sub> -compose.yml . . . . .                                            | 30        |
| 8.5.2     | Healthcheck . . . . .                                                                              | 30        |
| 8.5.3     | Variabili d'ambiente . . . . .                                                                     | 30        |
| 8.5.4     | Comandi . . . . .                                                                                  | 30        |
| <b>9</b>  | <b>Processo di sviluppo del Minimum Viable Product</b>                                             | <b>31</b> |
| 9.1       | Attività: sviluppo del Minimum Viable Product . . . . .                                            | 31        |
| 9.2       | Procedura: Implementazione dell'architettura frontend (Bulletproof React <sub>GG</sub> ) . . . . . | 31        |
| 9.3       | Procedura: sviluppo del backend stateless <sub>G</sub> . . . . .                                   | 31        |
| 9.4       | Strumenti e organizzazione repository <sub>G</sub> . . . . .                                       | 31        |
| <b>10</b> | <b>Processo di Verifica<sub>G</sub></b>                                                            | <b>33</b> |
| 10.1      | Attività: testing e validazione <sub>G</sub> continua . . . . .                                    | 33        |
| 10.2      | Procedura: Esecuzione test unitari . . . . .                                                       | 33        |
| 10.3      | Procedura: automazione quality gate (Pre-push) . . . . .                                           | 33        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>11 Processo di supporto (Qualità)</b>                               | <b>34</b> |
| 11.1 Attività: valutazione e selezione dei LLM . . . . .               | 34        |
| 11.2 Procedura: protocollo di testing comparativo e metriche . . . . . | 34        |

# 1 Introduzione

## 1.1 Scopo del documento

Questo documento nasce dall'esigenza di ottimizzare l'organizzazione interna del gruppo, avendo una distribuzione chiara dei compiti, una buona pianificazione delle attività e un controllo costante sui progressi durante tutta la durata del progetto. Vogliamo mantenere un ambiente di lavoro positivo e produttivo, basato sulla collaborazione, in modo da poter lavorare in maniera efficiente, risolvendo anche eventuali imprevisti riscontrati durante lo sviluppo. Crediamo nel miglioramento continuo: per questo ci impegniamo ad aggiornare e rivedere le nostre Norme di Progetto<sub>G</sub> ogni volta che si renda necessario, così da garantire che restino sempre allineate alle esigenze del progetto e alla crescita del team. Le Norme di Progetto<sub>G</sub> costituiscono infatti la formalizzazione del nostro Way of Working<sub>G</sub>, ovvero il modo in cui intendiamo collaborare e gestire le attività nel corso del progetto.

## 1.2 Cosa sono per noi le Norme di Progetto<sub>G</sub>?

Le Norme di Progetto<sub>G</sub> rappresentano il nostro modo di organizzare il lavoro nel progetto. È l'insieme di pratiche, strumenti e valori che il nostro team adotterà per realizzare le proprie attività. L'obiettivo è definire delle regole che possano garantire efficienza, efficacia<sub>G</sub> e collaborazione tra i membri del gruppo durante tutto il ciclo di vita del progetto.

## 2 Processi organizzativi

Questa sezione descrive i processi che regolano l'organizzazione interna del gruppo, la gestione delle attività e la comunicazione, sia interna che esterna. Ogni processo è descritto nelle sue attività, procedure operative e strumenti adottati.

### 2.1 Processo di gestione

Il processo di gestione ha lo scopo di pianificare, coordinare e controllare le attività del gruppo per tutta la durata del progetto.

#### 2.1.1 Attività: organizzazione dei ruoli

Il gruppo adotta una rotazione dei ruoli per garantire l'apprendimento completo di tutti i membri, una distribuzione equa delle responsabilità e lo sviluppo di competenze trasversali. I ruoli previsti sono:

- **Responsabile:** comunica con professori e azienda (tramite email), si occupa dell'approvazione finale di tutti i documenti e file, redige i verbali e si occupa dei diari di bordo;
- **Amministratore:** gestisce la repository<sub>G</sub> (crea e assegna le issue<sub>G</sub>, crea le milestone<sub>G</sub>); aggiorna il sito web del gruppo; si occupa della redazione dei documenti.
- **Analista:** svolge l'analisi dei requisiti<sub>G</sub> e dei casi d'uso.
- **Progettista:** progetta l'architettura del software; sceglie le tecnologie da utilizzare; scrive la documentazione di progetto (es. diagrammi UML); supporta i programmatori nell'implementazione.
- **Programmatore:** scrive il codice sorgente.
- **Verificatore:** controlla codice e documenti.

#### 2.1.2 Attività: pianificazione degli sprint<sub>G</sub>

Il lavoro è organizzato in sprint<sub>G</sub> secondo il framework Scrum<sub>G</sub>.

#### Procedure

- **Sprint Planning<sub>G</sub> (inizio sprint<sub>G</sub>):** il gruppo si riunisce per definire gli obiettivi dello sprint<sub>G</sub>, assegnare i ruoli e stimare le ore preventivate per ciascuna attività. Le attività vengono registrate come GitHub<sub>G</sub> Issues nella Project Board.
- **Sprint Review<sub>G</sub> (fine sprint<sub>G</sub>):** il gruppo verifica<sub>G</sub> il grado di raggiungimento degli obiettivi, confronta consuntivo e preventivo orario, identifica le attività non completate e le sposta nel backlog<sub>G</sub> o nello sprint<sub>G</sub> successivo.
- **Retrospettiva:** il gruppo analizza il processo adottato, individua le cause degli scostamenti rilevati, aggiorna l'analisi dei rischi con eventuali nuovi rischi emersi e definisce misure correttive per migliorare la pianificazione futura.

#### Durata degli sprint<sub>G</sub>

- **Fase RTB:** sprint<sub>G</sub> di 2 settimane.
- **Fase PB:** sprint<sub>G</sub> settimanali, per garantire un monitoraggio più frequente dell'avanzamento del prodotto.

## Strumenti

- **GitHub<sub>G</sub> Projects**: gestione delle issue<sub>G</sub> e monitoraggio dell'avanzamento tramite dashboard separate per fase RTB e PB.
- **Discord / Google Meet**: riunioni di planning e review.

## 2.2 Processo di comunicazione

Il processo di comunicazione definisce le modalità e gli strumenti adottati per garantire un flusso informativo efficace tra i membri del gruppo e con i soggetti esterni (proponente, committenti).

### 2.2.1 Attività: comunicazione interna

Ogni membro del gruppo si impegna a mantenere un atteggiamento aperto e disponibile, offrendo aiuto e supporto quando necessario.

**Procedure** Le comunicazioni interne si distinguono per urgenza e natura:

- Comunicazioni rapide e organizzative: messaggio su **WhatsApp**.
- Riunioni sincrone e sessioni di lavoro collaborativo: canale **Discord**.
- Riunioni formali e presentazioni: **Google Meet**.

## Strumenti

- **WhatsApp**: messaggistica rapida.
- **Discord**: videoconferenze interne e lavoro collaborativo.
- **Google Meet**: riunioni formali.

### 2.2.2 Attività: comunicazione esterna

Le comunicazioni con proponenti e committenti seguono canali formali.

## Procedure

- Comunicazioni formali scritte: **email**.
- Incontri periodici online con la proponente: **Google Meet**.
- Riunioni formali in presenza con la proponente: **incontri in presenza**, verbalizzati secondo le procedure definite nella sezione dedicata alla documentazione.

## Strumenti

- **Email**: comunicazioni ufficiali.
- **Google Meet**: incontri online con la proponente.

### 2.2.3 Attività: comunicazione nei periodi di vacanza/sessione

Il team si impegna a rimanere operativo durante i periodi di vacanza ed esami, privilegiando la comunicazione asincrona per gli aggiornamenti. I membri concorderanno obiettivi sprint<sub>G</sub> realistici e manterranno un dialogo costante. Questa evenienza è stata già considerata nell'analisi dei rischi in fase di candidatura, e sono state definite specifiche contromisure per mitigarne l'impatto.



## 2.3 Processo di Formazione

Il processo di formazione ha lo scopo di garantire che tutti i membri del gruppo acquisiscano le competenze tecniche e metodologiche necessarie allo svolgimento del progetto.

### 2.3.1 Attività: autoformazione individuale

Ciascun membro è responsabile dello studio autonomo delle tecnologie e degli strumenti adottati dal gruppo.

#### Procedure

- Ogni membro studia in modo autonomo le tecnologie assegnate al proprio ruolo nello sprint<sub>G</sub> corrente, in particolare nei primi sprint<sub>G</sub> delle fasi principali del progetto.
- Le difficoltà incontrate vengono segnalate durante le riunioni di planning o review, attivando eventualmente una sessione di supporto collettivo.
- Le risorse di riferimento (documentazione ufficiale, slide del corso, tutorial) vengono condivise nel canale Discord dedicato o su Google Drive.

#### Strumenti

- **Discord**: condivisione di risorse e supporto tra pari.
- **Google Drive**: condivisione di documenti e appunti.
- **Documentazione ufficiale** degli strumenti adottati (React<sub>G</sub>, Flask<sub>G</sub>, PostgreSQL<sub>G</sub>, ecc.).
- **Slide del corso** di Ingegneria del Software.

## 3 Gestione del codice e documentazione

Il gruppo adotta una strategia multi-repository<sub>G</sub> su *GitHub*<sub>G</sub> per separare la gestione documentale dallo sviluppo tecnico:

- **ByteMe**: Repository<sub>G</sub> dedicata esclusivamente alla documentazione ufficiale del progetto (Norme, Piano di Progetto, Analisi, ecc.) in formato LaTeX.
- **POC\_ByteMe**: Repository<sub>G</sub> utilizzata durante la fase RTB per lo sviluppo del Proof of Concept, finalizzata alla validazione<sub>G</sub> tecnologica e all'integrazione con i modelli LLM.
- **Second\_Brain**: Repository<sub>G</sub> dedicata allo sviluppo del prodotto finale (MVP) durante la fase di PB. Qui verrà implementata l'architettura definitiva basata su React<sub>G</sub>.

### 3.1 Processo di gestione della documentazione

#### 3.1.1 Attività: organizzazione della documentazione

L'attività di organizzazione della documentazione ha lo scopo di strutturare in modo chiaro e coerente tutti i documenti prodotti dal gruppo durante il progetto. Una corretta organizzazione consente di garantire ordine e separazione logica tra i diversi materiali, facilitando il lavoro del gruppo e anche la consultazione di persone esterne.

#### Procedure

- La documentazione viene divisa in base alle fasi del progetto, cioè vogliamo distinguere i documenti per la Candidatura, per la RTB e per la PB.
- Vogliamo inoltre classificare i documenti in base alla loro destinazione distinguendo quindi tra documenti ufficiali destinati a revisione e consegna esterna e documenti interni di supporto al lavoro del gruppo.
- I verbali delle riunioni verranno divisi in interni (riunioni interne al gruppo) ed esterni (riunioni con l'azienda proponente).
- Infine vogliamo mantenere separati i file sorgenti dei documenti (in formato LaTeX) dai file compilati (in formato PDF), per garantire una gestione ordinata delle modifiche.

**Strumenti** In seguito ai ragionamenti sopra citati, abbiamo strutturato come segue la repository<sub>G</sub> Github<sub>G</sub>:

```
ByteMe/  
|-- .gitignore  
|-- 1_Candidatura/  
|-- 2_RTb/  
|-- 3_PB/  
|-- Documenti_interni/  
|-- docs/  
|-- scripts/  
|-- src/  
|-- Verbali_esterni/  
|-- Verbali_interni/
```

Ogni cartella contiene quindi tutti i documenti relativi alle rispettive fasi del progetto. La cartella dei Documenti interni contiene le Norme di Progetto<sub>G</sub> (ovvero il documento Way of working<sub>G</sub>) e il Glossario<sub>G</sub>. La cartella docs contiene i file relativi al sito web del gruppo, dove

sono esposti i documenti in pdf. La cartella `src` contiene i file `.tex` di tutti i documenti del gruppo, che vengono convertiti in `.pdf` nelle cartelle corrette al momento del push sulla repository<sub>G</sub>. Per rendere automatico questo processo si usa il file `compileLatex.yml` che si trova nella cartella `.githubG/workflows`. Le cartelle relative ai verbali contengono tutti i verbali in pdf relativi alle riunioni interne del gruppo o agli incontri con la proponente.

### 3.1.2 Attività: sviluppo della documentazione

L'attività di sviluppo della documentazione comprende tutte le operazioni necessarie alla creazione, modifica e aggiornamento dei documenti di progetto. Si vuole garantire tracciabilità delle modifiche, coerenza tra i contenuti e allineamento tra i membri del gruppo.

#### Procedure

- Sincronizzazione iniziale: prima di iniziare qualsiasi modifica, ogni membro aggiorna la propria copia locale della repository<sub>G</sub> per lavorare sull'ultima versione disponibile.
- Modifica dei documenti: i documenti vengono modificati localmente seguendo le convenzioni stabilite (nel dettaglio in strumenti).
- Descrizione delle modifiche: ogni modifica viene registrata tramite `commitG`, accompagnato da un messaggio descrittivo che ne riassume il contenuto.
- Condivisione delle modifiche: le modifiche vengono caricate sulla repository<sub>G</sub> remota, rendendole disponibili agli altri membri del gruppo.
- Automazione della compilazione: ad ogni aggiornamento della repository<sub>G</sub>, viene eseguita automaticamente la compilazione dei documenti per garantire la correttezza del formato finale.

#### Strumenti

- Sincronizzazione iniziale: Prima di iniziare a lavorare, aggiornare sempre la repository<sub>G</sub> locale con il comando:

```
git pull origin main
```

- Modifica dei documenti: i documenti devono essere modificati dal corrispondente file `.tex` nella cartella `src/`. Per i file multimediali (loghi, immagini) ricordarsi di utilizzare percorsi relativi:

```
\includegraphics{../logo.png}
```

- Descrizione e condivisione delle modifiche: utilizzare la seguente sequenza di comandi per pubblicare le modifiche:

```
git add .
git commit -m "Descrizione chiara delle modifiche effettuate"
git push origin main
```

- Automazione della compilazione: il repository<sub>G</sub> è configurato con un sistema di automazione che gestisce la compilazione dei documenti:

- **File di configurazione:** `compileLatex.yml`
- **Funzionalità:** Compila automaticamente i file `.tex` in `.pdf` dopo ogni push
- **Destinazione:** I PDF generati vengono salvati nelle cartelle di destinazione appropriate
- **Vantaggi:**
  - \* Compilazione automatica e consistente
  - \* Riduzione degli errori di compilazione manuale
  - \* Tracciabilità delle versioni PDF

### 3.1.3 Attività: `versionamentoG` della documentazione

Per ogni documento, eccetto i verbali, vengono registrate le modifiche apportate nella tabella "Registro dei Cambiamenti". Come convenzione per il `versionamentoG` è stato adottato il formato **SEMVER** (Semantic Versioning). Ad ogni modifica, la versione del documento viene aggiornata secondo il formato `x.y.z`, dove:

- `x` (major): indica modifiche sostanziali
- `y` (minor): indica aggiunte di funzionalità
- `z` (patch): indica correzioni di errori

### 3.1.4 Attività: `verificaG` della documentazione

L'attività di `verificaG` della documentazione ha lo scopo di garantire la correttezza, la coerenza e la completezza dei documenti prodotti dal gruppo. Essa permette di individuare eventuali errori, ambiguità o incoerenze prima dell'approvazione finale, assicurando un livello qualitativo adeguato.

#### Procedure

- `VerificaG` incrociata: ogni documento viene verificato da uno o più membri del gruppo diversi dagli autori originali, secondo il principio della "`verificaG` incrociata".
- Controllo dei contenuti: i verificatori esaminano il documento per assicurare:
  - Correttezza grammaticale e di contenuto;
  - Completezza delle informazioni;
  - Aderenza agli standard definiti nelle Norme di Progetto<sub>G</sub>.
- Modifiche ai documenti: ogni modifica effettuata per correggere o integrare i documenti viene tracciata nella tabella all'inizio di ogni documento.
- Approvazione finale: il documento viene considerato approvato quando:
  - Tutte le verifiche sono state superate;
  - Il responsabile di progetto fornisce l'approvazione finale.

#### Strumenti

- `GitHubG` per la condivisione e il `versionamentoG` dei documenti;
- Sistema di gestione delle versioni per tracciare le modifiche;
- Script automatici (ad esempio il file `gulpease.py`).

### 3.1.5 Sito web della documentazione

Il gruppo ha implementato un sito web per ospitare la documentazione del progetto, accessibile all'indirizzo: <https://byte25.github.io/ByteMe/>

Il sito, generato automaticamente dalla repository<sub>G</sub> GitHub<sub>G</sub>, offre i seguenti benefici:

- **Versionamento<sub>G</sub>**: mantenimento dello storico delle versioni dei documenti
- **Accesso centralizzato**: punto unico di accesso per tutta la documentazione
- **Responsive design**: consultabile da qualsiasi dispositivo

La pubblicazione avviene tramite GitHub<sub>G</sub> Pages, assicurando che la documentazione online sia sempre aggiornata all'ultima versione approvata.

## 3.2 Processo di gestione del codice

Il processo di gestione del codice definisce le modalità con cui il gruppo sviluppa, aggiorna e monitora il codice sorgente del progetto. L'obiettivo è garantire tracciabilità delle modifiche, collaborazione tra i membri del team e controllo dell'avanzamento delle attività di sviluppo.

### 3.2.1 Attività: gestione delle attività di sviluppo

L'attività di gestione delle attività di sviluppo ha lo scopo di organizzare, tracciare e monitorare i compiti relativi allo sviluppo del codice, garantendo una distribuzione chiara delle responsabilità e un controllo costante dell'avanzamento del lavoro.

#### Procedure

- **Definizione delle attività**: ogni attività di sviluppo viene formalizzata come unità di lavoro, descritta in modo chiaro e assegnata a uno o più membri del team.
- **Tracciamento delle attività**: le attività vengono identificate tramite un riferimento univoco (ad esempio un numero), che ne permette il monitoraggio durante tutto il ciclo di sviluppo.
- **Collegamento con i verbali**: le decisioni prese durante le riunioni vengono collegate alle attività operative, garantendo coerenza tra pianificazione e sviluppo effettivo.
- **Monitoraggio dello stato di avanzamento**: le attività vengono aggiornate in base al loro stato (da fare, in corso, completate), permettendo al team di avere una visione chiara dell'avanzamento complessivo.

**Strumenti** Per una gestione efficiente e tracciabile delle attività di progetto, il gruppo utilizza le **GitHub<sub>G</sub> Issues** come strumento principale. Ogni attività viene registrata come Issue<sub>G</sub> nella repository<sub>G</sub> e assegnata un **numero identificativo univoco** generato automaticamente da GitHub<sub>G</sub>. Le Issue<sub>G</sub> vengono categorizzate tramite label specifiche e assegnate ai membri del team responsabili.

Nei verbali interni, nella sezione "Azioni da Intraprendere", viene fatto esplicito riferimento alle Issue<sub>G</sub> correlate tramite il loro numero identificativo (#numero). Questo approccio garantisce un tracciamento completo dalle decisioni dei verbali alle attività concrete da svolgere.

Inoltre per organizzare meglio il flusso di lavoro, sono state create due dashboard distinte per le Issue<sub>G</sub>:

- **Dashboard RTB**: dedicata alla prima fase del progetto, comprendente analisi dei requisiti<sub>G</sub>, definizione dei casi d'uso, definizione architettura e scelta tecnologie

- **Dashboard PB:** riservata alla seconda fase, focalizzata sullo sviluppo del prodotto

Questa suddivisione permette di monitorare separatamente lo stato di avanzamento delle due fasi principali del progetto, garantendo che tutte le attività vengano completate entro le scadenze stabilite per ciascuna milestone<sub>G</sub>.

### 3.2.2 Attività: metodologia Git<sub>G</sub> Flow

Per la gestione dei rami (branching strategy), il gruppo adotta il modello **Git<sub>G</sub> Flow**. Questa metodologia permette di isolare lo sviluppo di nuove funzionalità, gestire i rilasci e correggere bug critici in modo ordinato.

#### Rami principali

- **main:** contiene esclusivamente codice stabile e pronto per la produzione. Ogni push su questo ramo corrisponde a una release ufficiale.
- **develop:** è il ramo principale di sviluppo. Raccoglie tutte le funzionalità completate in attesa del prossimo rilascio.

#### Rami di supporto

- **feature/:** utilizzati per lo sviluppo di nuove funzionalità o refactoring. Nascono da **develop** e tornano in **develop** una volta terminati.
- **release/:** rami temporanei per la preparazione di una nuova versione (es. **release/1.0.0**). Servono per bugfix dell'ultimo minuto e aggiornamento metadati.
- **hotfix/:** rami per correzioni urgenti da applicare direttamente alla produzione. Nascono da **main** e vengono uniti sia a **main** che a **develop**.

**Procedure operative** Il gruppo utilizza l'estensione **git<sub>G</sub>-flow** per automatizzare i passaggi. La sequenza standard per una nuova funzionalità è:

1. **Inizio:** `gitG flow feature start nome.feature`
2. **Sviluppo:** `commitG` regolari sul ramo isolato
3. **Conclusione:** `gitG flow feature finish nome.feature` (esegue il `mergeG` su **develop** e pulisce il ramo locale)

### 3.2.3 Attività: automazione dei test pre-push

Al fine di garantire che il ramo **develop** rimanga sempre in uno stato integro e funzionante, il gruppo adotta una politica di **blocco del push** in caso di test falliti.

**Procedure** Viene utilizzato un **Git<sub>G</sub> Hook** di tipo **pre-push**. Si tratta di uno script locale che intercetta il comando di caricamento sul server e avvia la suite di test.

- **Esecuzione in Docker<sub>G</sub>:** Per garantire la parità tra gli ambienti di sviluppo (parità tra il PC locale e il container), lo script esegue i test tramite `docker composeG run`.
- **Vincolo di successo:** Se anche un solo test fallisce, il push viene interrotto forzatamente da Git<sub>G</sub>.

**Strumenti e configurazione locale** Ogni membro del team è tenuto a configurare il proprio ambiente seguendo queste istruzioni:

1. Rendere eseguibile lo script di controllo: `chmod +x scripts/run_tests.sh`.
2. Collegare l'hook locale: Creare il file `.git/hooks/pre-push` contenente la chiamata allo script e renderlo eseguibile (`chmod +x`).

### 3.2.4 Attività: gestione delle dipendenze (`requirements.txt`)

Per il backend, il gruppo mantiene un file `requirements.txt` snello, limitato alle librerie strettamente necessarie per un'architettura *stateless<sub>G</sub>*. Durante la fase PB, sono obbligatorie le seguenti dipendenze per il corretto funzionamento dei test:

- `pytestG` e `pytestG-flaskG` per l'automazione.
- `flaskG`, `flaskG-cors`, `openai`, `pydantic`, `python-dotenv` per la logica applicativa.

## 4 Documenti

All'interno della repository<sub>G</sub> di progetto sulla documentazione è presente una serie di cartelle e documenti strutturati con l'obiettivo di organizzare in modo ordinato i materiali prodotti dal gruppo. Di seguito si riporta una descrizione delle principali sezioni e del loro contenuto.

### 4.1 .github/workflows

Questa cartella contiene gli script di automazione utilizzati per la compilazione automatica dei documenti LaTeX tramite GitHub Actions. Contiene il file `compileLatex.yml`.

### 4.2 Candidatura

Questa cartella racchiude tutti i file pdf relativi alla fase di candidatura del gruppo al progetto. Include la lettera di presentazione, il documento sul preventivo dei costi e dell'impegno orari e la valutazione di tutti i capitolati.

### 4.3 RTB

Questa cartella contiene tutti i documenti pdf relativi alla RTB (Requirements and Technology Baseline), ovvero raccoglie tutto ciò che costituisce la prima baseline del progetto. Sono presenti:

- **Analisi dei Requisiti<sub>G</sub>:** Questo documento descrive l'analisi dei requisiti<sub>G</sub> per il progetto "Second Brain". Vengono quindi descritti i casi d'uso, e per ciascuno, i rispettivi requisiti funzionali<sub>G</sub> o non funzionali.
- **Piano di Progetto:** Questo documento ha lo scopo di definire la pianificazione, l'organizzazione e il controllo delle attività del progetto Second Brain. Viene quindi fornita una sintesi delle attività, dei risultati e dei progressi conseguiti in ogni Sprint<sub>G</sub>.
- **Piano di Qualifica:** Questo documento ha lo scopo di definire gli standard di qualità, le metriche di processo e di prodotto e le procedure di verifica<sub>G</sub> che il gruppo deve rispettare. Contiene il "cruscotto di valutazione" per monitorare l'andamento del progetto rispetto agli obiettivi prefissati.

### 4.4 PB

Questa cartella contiene tutti i documenti pdf relativi alla PB (Product Baseline), ovvero raccoglie tutto ciò che costituisce la seconda baseline del progetto. Sono presenti:

- **Specifica Tecnica<sub>G</sub>:** Documento ad uso interno del team di sviluppo che dettaglia l'architettura software del prodotto. Descrive i design pattern adottati, la scomposizione dei componenti basati sul framework React<sub>G</sub>, la struttura del database<sub>G</sub> per la persistenza delle informazioni e le logiche di integrazione con le API<sub>G</sub> degli LLM (Large Language Models).
- **Manuale Utente<sub>G</sub>:** Documento destinato agli utilizzatori finali che illustra le procedure di installazione, configurazione e utilizzo dell'applicazione. Fornisce istruzioni operative per l'editing dei file, la gestione del sistema "Second Brain" e l'interazione con le funzionalità di assistenza generativa offerte dall'Intelligenza Artificiale.



## 4.5 Documenti interni

Questa cartella contiene i documenti pdf redatti dal gruppo per l'organizzazione interna delle attività. Sono presenti:

- **Glossario<sub>G</sub>**: questo documento ha lo scopo di raccogliere i termini utilizzati nella documentazione del gruppo ByteMe durante il progetto.
- **Norme di Progetto<sub>G</sub>**: questo documento stabilisce regole, procedure e strumenti per organizzare il lavoro del gruppo. Include anche il Way of Working<sub>G</sub>, descrivendo ruoli, responsabilità, riunioni e modalità di collaborazione per garantire efficienza e qualità.

## 4.6 Verbalì esterni

Questa cartella contiene i file pdf dei verbalì delle riunioni con i rappresentanti dell'azienda proponente. Vengono documentati confronti, chiarimenti e decisioni concordate.

## 4.7 Verbalì interni

Questa cartella contiene i file pdf dei verbalì delle riunioni svolte tra i membri del gruppo. Vengono documentate decisioni prese, pianificazioni e attività interne.

## 4.8 docs

Questa cartella contiene tutti i file relativi al nostro sito web, in particolare:

- **assets**: cartella contenente tutte le immagini.
- **css**: cartella contenente il main.css, per lo stile del sito.
- **js**: cartella contenente i file .js.
- **files.html**: file html della sezione "tutti i file" del sito.
- **index.html**: file html della home del sito.

## 4.9 scripts

Questa cartella contiene degli scripts, quali uno script (gulpease.py) per calcolare l'indice di leggibilità di un testo, un correttore automatico (spellcheck.py) e un file whitelist.txt che contiene l'elenco di tutte le parole da ignorare per la correzione ortografica. Inoltre è presente glossario<sub>G</sub>.py, che permette di mettere automaticamente le G al pedice delle parole nel glossario<sub>G</sub>.

## 4.10 src

Questa cartella contiene i file sorgenti LaTeX, divisi nelle rispettive cartelle (omonime a quelle contenenti i pdf relativi).

## 4.11 README.md

Questo è il file che presenta il gruppo e la repository<sub>G</sub>.

## 5 Analisi dei Requisiti<sub>G</sub> (AdR)

L'**analisi dei requisiti<sub>G</sub>** rappresenta una fase fondamentale del processo di sviluppo, in quanto permette di identificare gli obiettivi del sistema, le funzionalità attese e i vincoli da rispettare. Tale attività costituisce il fondamento su cui vengono impostate le successive fasi di progettazione e implementazione. Il gruppo ha adottato un approccio collaborativo finalizzato a garantire una definizione adeguata dei requisiti.

### 5.1 Studio dei casi d'uso

#### 5.1.1 Attività: identificazione iniziale dei casi d'uso

L'attività di identificazione iniziale dei casi d'uso ha lo scopo di individuare le principali funzionalità del sistema e costruire una visione condivisa del progetto.

**Procedure** Questa prima fase dell'analisi ha previsto un insieme di riunioni dedicate alla raccolta di tutti i possibili use case, utilizzando la tecnica del brainstorming. Durante questi incontri il team ha:

- Discusso le esigenze principali del sistema;
- Ipotizzato diversi casi d'uso e potenziali funzionalità;
- Chiarito dubbi e definito una struttura condivisa per la descrizione dei requisiti e dei casi d'uso, così da evitare ambiguità.

Questa fase ha permesso di costruire una visione comune del dominio applicativo e di delineare una prima struttura dei casi d'uso principali.

**Strumenti** I principali strumenti utilizzati in questa fase sono stati:

- Discord: piattaforma scelta per le riunioni di gruppo.
- Github<sub>G</sub>: per la raccolta delle informazioni.

#### 5.1.2 Attività: analisi individuale dei casi d'uso

Ogni caso d'uso viene analizzato in dettaglio da un membro del gruppo per garantire una descrizione completa e accurata.

**Procedure** Una volta definita la lista di tutti i casi d'uso possibili, ogni membro del gruppo ha assunto la responsabilità di analizzarne uno in maniera approfondita. In particolare, ognuno ha:

- Studiato nel dettaglio il caso d'uso assegnato;
- Definito attori, precondizioni, postcondizioni, scenario principale e eventuali estensioni;
- Qualora necessario, studiato nel dettaglio eventuali sottocasi.

Questa suddivisione del lavoro ha permesso di svolgere un'analisi più accurata e di sfruttare in modo efficiente le competenze di ogni membro del gruppo.

**Strumenti**

- Github<sub>G</sub>: per la raccolta delle informazioni.
- LaTeX: per la scrittura dei casi d'uso.

### 5.1.3 Attività: revisione dei casi d'uso

I casi d'uso vengono sottoposti a verifica<sub>G</sub> incrociata per garantirne coerenza e qualità.

**Procedure** Una fase essenziale del processo è stata la revisione incrociata dei contenuti prodotti individualmente. Ogni membro ha esaminato il lavoro degli altri al fine di:

- Verificare la coerenza rispetto alle decisioni emerse nelle riunioni iniziali;
- Controllare l'aderenza agli standard interni stabiliti nella prima fase;
- Individuare eventuali incoerenze, ridondanze o mancanze;
- Migliorare la chiarezza e la completezza delle descrizioni.

Il confronto tra i membri del gruppo ha contribuito a uniformare lo stile, a garantire la qualità complessiva dell'analisi e a minimizzare il rischio di errori o interpretazioni divergenti.

#### Strumenti

- Github<sub>G</sub>: per la raccolta delle informazioni.
- LaTeX: per la scrittura dei casi d'uso.

### 5.1.4 Attività: discussione dei casi d'uso con l'azienda

I casi d'uso vengono presentati all'azienda per validare le scelte effettuate.

**Procedure** Il 2 dicembre 2025 il gruppo ha svolto anche una riunione dedicata all'analisi dei casi d'uso con l'azienda proponente. Durante l'incontro sono stati presentati gli scenari elaborati dal team, con l'obiettivo di confrontare i casi d'uso individuati internamente e verificarne la correttezza rispetto alle reali esigenze del cliente. Questo confronto ha permesso di validare le ipotesi iniziali, correggere eventuali interpretazioni errate e allineare le aspettative del progetto tra gruppo e azienda.

#### Strumenti

- Riunione in presenza con l'azienda proponente.

## 5.2 Studio dei requisiti

### 5.2.1 Attività: analisi individuale dei requisiti

I requisiti vengono definiti a partire dai casi d'uso validati, garantendo chiarezza e tracciabilità.

**Procedure** Una volta definiti e validati i casi d'uso, il gruppo ha proceduto alla definizione dei requisiti, assegnando a ciascun membro la responsabilità di redigerne una parte. In particolare, ogni membro ha:

- Scritto i requisiti partendo dai casi d'uso analizzati;
- Formulato i requisiti in modo semplice ma chiaro;
- Utilizzato una dicitura condivisa per identificare in modo univoco i requisiti, distinguendo tra requisiti obbligatori (O), desiderabili (D) e opzionali (OP);

- Classificato ogni requisito in base alla sua tipologia, distinguendo tra requisiti funzionali<sub>G</sub> (F), di qualità (Q) e di vincolo (V).

Questa organizzazione del lavoro e l'adozione di una notazione comune hanno permesso di ottenere un insieme di requisiti coerente, uniforme e facilmente comprensibile, permettendo inoltre la verifica<sub>G</sub> e eventuale correzione ulteriore dei casi d'uso precedentemente scritti.

### Strumenti

- Github<sub>G</sub>: per la raccolta delle informazioni.
- LaTeX: per la scrittura dei casi d'uso.

### 5.2.2 Attività: revisione dei requisiti

L'attività di revisione dei requisiti garantisce la qualità dei requisiti attraverso verifica<sub>G</sub> incrociata, assicurando coerenza con i casi d'uso, completezza e chiarezza delle specifiche.

**Procedure** I membri del gruppo si sono impegnati ad esaminare tutti i requisiti al fine di:

- Verificare la coerenza con i casi d'uso precedentemente definiti;
- Controllare la completezza dei requisiti rispetto alle funzionalità previste;
- Individuare eventuali requisiti mancanti, ridondanti o poco chiari;
- Migliorare la chiarezza e la precisione delle formulazioni dei requisiti.

Il confronto tra i membri del gruppo ha contribuito a rafforzare la qualità complessiva del documento dei requisiti e a ridurre il rischio di errori, mancanze o ridondanze.

### Strumenti

- Github<sub>G</sub>: per la raccolta delle informazioni.
- LaTeX: per la scrittura dei casi d'uso.

## 5.3 Scrittura del documento definitivo

Al termine delle fasi precedenti, il team ha proceduto alla redazione del documento “Analisi dei Requisiti<sub>G</sub>”, nel quale tutti i casi d'uso e requisiti sono stati integrati e uniformati in un'unica struttura coerente. Il risultato è un documento chiaro, consistente e verificabile.

## 5.4 Giudizio sull'approccio adottato

L'approccio adottato durante l'analisi dei requisiti<sub>G</sub> ha contribuito in modo significativo alla crescita del team, sia sul piano tecnico sia su quello organizzativo. Grazie alle attività svolte, il gruppo ha:

- Imparato a strutturare i casi d'uso in maniera chiara e coerente, anche dopo un iniziale disallineamento nello stile di documentazione che ha richiesto un successivo lavoro di uniformazione;
- Acquisito maggiore familiarità con i processi di revisione incrociata, migliorando la capacità di individuare errori, ambiguità o parti mancanti nel lavoro dei colleghi;
- Migliorato le abilità di collaborazione e comunicazione all'interno del gruppo;
- Sviluppato una migliore capacità di analizzare i requisiti in modo critico.

## 6 Piano di Progetto (PdP)

Il **Piano di Progetto** costituisce uno strumento fondamentale per organizzare, monitorare e coordinare le attività del team durante l'intero ciclo di sviluppo. Esso definisce la pianificazione temporale, la distribuzione dei ruoli, il carico di lavoro dei membri tramite un riassunto di ogni sprint<sub>G</sub> e l'analisi dei potenziali rischi, garantendo una gestione strutturata e consapevole del progetto.

### 6.1 Attività: analisi dei rischi

Un elemento centrale del Piano di Progetto è l'analisi dei rischi, svolta con l'obiettivo di identificare gli eventi critici che avrebbero potuto compromettere tempi, qualità o continuità del lavoro.

**Procedure** L'analisi è stata condotta esaminando individualmente ciascun potenziale problema, classificandolo all'interno di tre categorie principali: rischi organizzativi, rischi tecnologici e rischi legati al prodotto. Per ogni rischio sono stati definiti gli elementi essenziali per una valutazione completa:

- Una descrizione chiara del problema;
- La probabilità che succedano;
- La gravità del problema;
- Le misure di prevenzione;
- Le misure di mitigazione.

In questo modo abbiamo potuto capire meglio i possibili problemi e prepararci a gestirli in ogni sprint<sub>G</sub>.

#### Strumenti

- Discord: piattaforma scelta per le riunioni di gruppo.
- Github<sub>G</sub>: per la raccolta delle informazioni.
- LaTeX: per la scrittura dei casi d'uso.

### 6.2 Metodo adottato

Il gruppo ha deciso di utilizzare la metodologia Agile<sub>G</sub>, in particolare il framework Scrum<sub>G</sub>, che prevede una pianificazione continua e iterativa del lavoro. Fin dall'inizio sono stati definiti i ruoli di progetto, assegnati e alternati tra i membri del team nei diversi Sprint<sub>G</sub> per garantire una distribuzione equilibrata delle responsabilità e favorire la crescita delle competenze. La pianificazione delle attività è stata gestita Sprint<sub>G</sub> per Sprint<sub>G</sub>, adattandosi ai progressi del team e ai risultati ottenuti.

### 6.3 Attività: pianificazione e gestione degli sprint<sub>G</sub>

L'attività di pianificazione e gestione degli sprint<sub>G</sub> consente di organizzare il lavoro definendo obiettivi, ruoli e attività e monitorandone l'avanzamento.

## Procedure

- Definizione degli obiettivi all'inizio di ogni  $\text{sprint}_G$ .
- Assegnazione dei ruoli di progetto tra i membri del team con rotazione dei ruoli ad ogni  $\text{sprint}_G$ .
- Pianificazione e assegnazione delle attività e stima delle ore necessarie.
- Identificazione dei rischi associati alle attività pianificate.
- Aggiornamento della pianificazione in base all'andamento del progetto.

## Strumenti

- Discord: piattaforma scelta per le riunioni di gruppo.
- Github<sub>G</sub> Issues: per l'assegnazione delle attività.
- Github<sub>G</sub> Projects: per l'organizzazione delle issues (fasi RTB e PB).

### 6.4 Attività: consuntivazione degli $\text{sprint}_G$

Al termine di ogni  $\text{sprint}_G$ , il gruppo analizza le attività svolte per valutare i risultati ottenuti, individuare eventuali criticità e migliorare il processo nei cicli successivi.

**Procedure** A conclusione di ogni  $\text{Sprint}_G$ , il gruppo ha redatto un resoconto delle attività svolte, documentando le sue fasi:

- Pianificazione: con obiettivi, ruoli, rischi e ore occupate previsti;
- Review: con attività e ore svolte e eventuali problemi riscontrati;
- Retrospettiva: con eventuali dubbi o problemi e la soluzione trovata, e eventuali attività programmate per lo  $\text{sprint}_G$  successivo.

## Strumenti

- Discord: piattaforma scelta per le riunioni di gruppo.
- Github<sub>G</sub>: per la raccolta delle informazioni.

## 7 Piano di Qualifica (PdQ)

Il **Piano di Qualifica** è il documento che definisce la strategia del gruppo per garantire la qualità del prodotto software e l'efficienza dei processi di sviluppo. Rappresenta il riferimento principale per tutte le attività di Verifica<sub>G</sub> e Validazione<sub>G</sub>.

### 7.1 Scopo del documento

- **Pianificazione:** Fissare *ex-ante* gli standard qualitativi da raggiungere. Definisce quali metriche utilizzare, come calcolarle e quali sono le soglie di accettabilità (minime) e di ottimalità (desiderabili).
- **Consuntivo (Cruscotto):** Riportare *ex-post* i risultati delle misurazioni effettuate durante le varie fasi di progetto, permettendo di valutare oggettivamente lo stato di salute del progetto e la conformità del prodotto rispetto ai requisiti.

### 7.2 Contenuto e struttura

Il documento è strutturato per rispondere alle norme di qualità ISO/IEC 9126 e ISO/IEC 25010 ed è suddiviso nelle seguenti macro-aree:

- **Qualità di processo:** Definisce le metriche per monitorare l'andamento dei lavori, inclusi la gestione del budget, la pianificazione temporale e la stabilità dei requisiti.
- **Qualità di prodotto:** Definisce le metriche per valutare il software prodotto, coprendo aspetti quali l'adeguatezza funzionale<sub>G</sub>, l'efficienza, l'usabilità, l'affidabilità e la manutenibilità.
- **Strategia di verifica<sub>G</sub> e testing:** Descrive le procedure operative per l'Analisi Statica<sub>G</sub> e l'Analisi Dinamica<sub>G</sub> (livelli di test: Unit, Integration, System, Acceptance).
- **Cruscotto di qualità<sub>G</sub>:** Una sezione dinamica che viene aggiornata ad ogni revisione di progetto, contenente i grafici e le tabelle con i valori effettivi misurati per ogni metrica, evidenziando eventuali non conformità e le relative azioni correttive intraprese.

### 7.3 Metodologia di calcolo dell'Earned Value (EV)

Al fine di monitorare oggettivamente l'avanzamento del progetto ed evitare stime soggettive, il gruppo adotta la tecnica di misurazione fisica 0/100 (All-or-Nothing). Il calcolo avviene secondo le seguenti regole:

1. Ogni attività (Task/Issue<sub>G</sub>) pianificata nello Sprint<sub>G</sub> ha un valore in ore associato, definito nel Piano di Progetto.
2. Al termine dello Sprint<sub>G</sub>, o nei momenti di verifica<sub>G</sub> intermedi, si controlla lo stato di ciascuna attività.
3. Il valore dell'attività viene sommato all'Earned Value (EV) totale se e solo se l'attività soddisfa interamente la Definition of Done (DoD).
4. Se un'attività è parzialmente completa (es. codice scritto ma non revisionato o verificato), il suo contributo all'EV è 0.

## 7.4 Definition of Done (DoD)

Affinché un'unità di lavoro (Task/Issue<sub>G</sub>) possa essere considerata "Fatta" (Done) e conteggiata nel calcolo dell'Earned Value, deve soddisfare i seguenti criteri:

- **Codice:** Il codice sorgente è stato scritto, committato e pushato sul repository<sub>G</sub> ufficiale nel branch corretto.
- **Documentazione:** Il codice è corredato di commenti ed eventuali modifiche o scelte tecniche sono state riportate nella documentazione ufficiale.
- **Analisi Statica<sub>G</sub>:** Il codice supera i controlli di stile e qualità senza errori o warning bloccanti.
- **Revisione:** Il lavoro svolto è stato revisionato e verificato da uno o più membri del gruppo.

## 7.5 Strumenti e metriche di qualità automatizzate

Al fine di garantire un monitoraggio continuo e oggettivo della qualità, il gruppo ha integrato nel processo di sviluppo una serie di script automatici eseguiti tramite GitHub<sub>G</sub> Actions. Di seguito vengono descritte le metriche monitorate e il funzionamento degli strumenti adottati.

### 7.5.1 Qualità della documentazione

#### Correttezza Ortografica (CO)

- **Descrizione e scopo:** Misura il numero di errori ortografici di battitura presenti nei documenti. L'obiettivo è garantire professionalità e assenza di refusi che potrebbero compromettere la comprensione del testo.
- **Calcolo:** Conteggio ed esposizione delle parole non presenti nel dizionario italiano standard e non incluse nella `whitelist` di progetto (che contiene acronimi, nomi propri e termini tecnici accettati).
- **Automazione:** Uno script Python (`scripts/spellcheck.py`) analizza ricorsivamente tutti i file `.tex`. Lo script rimuove i comandi LaTeX (es. `\section`, `\textbf`) per analizzare solo il testo puro. Le parole sconosciute vengono confrontate con la libreria `pyspellchecker` e, se non presenti nella `whitelist`, vengono segnalate come errori. Le parole individuate vanno controllate e corrette nei documenti per evitare errori ortografici.

#### Indice di Gulpease (IG)

- **Descrizione e scopo:** È un indice di leggibilità calibrato sulla lingua italiana. Valuta quanto un testo sia comprensibile in base alla lunghezza delle parole e delle frasi.
- **Calcolo:** La formula utilizzata è:

$$89 + \frac{300 \times (\text{numero frasi}) - 10 \times (\text{numero lettere})}{\text{numero parole}}$$

- **Automazione:** Uno script Python (`scripts/gulpease.py`) analizza i file sorgente, calcolando i totali di lettere, parole e frasi (riconoscendo la punteggiatura di fine periodo). Restituisce un valore medio per documento, che deve essere superiore a 40. Se tutti i documenti hanno un IG adeguato, il badge presente nel README della repo della documentazione diventa verde; in caso contrario diventa rosso e i file vanno rivisitati.



### 7.5.2 Qualità del codice

#### Lines of Code (LOC)

- **Descrizione e Scopo:** Rappresenta la dimensione fisica del software misurata in righe di codice. Serve a monitorare la crescita del progetto e a stimare lo sforzo di manutenzione.
- **Calcolo:** Conteggio delle righe non vuote e non di commento nei file sorgente.
- **Automazione:** Viene utilizzato il tool `cloc` (Count Lines of Code) all'interno della pipeline `GitHubG`. Lo strumento è configurato per escludere file generati automaticamente, immagini, librerie esterne (es. `node_modules`) e file di configurazione, fornendo un conteggio fedele del lavoro svolto dal team. Questa metrica non è vincolante ma permette di monitorare la dimensione dei sorgenti.

#### Complessità Ciclomatica (McCabe)

- **Descrizione e scopo:** Misura la complessità logica di una funzione o metodo contando il numero di cammini linearmente indipendenti attraverso il codice sorgente. Una complessità elevata indica codice difficile da testare e mantenere.
- **Calcolo:** Basato sul grafo di flusso di controllo del programma. Semplificando, incrementa il conteggio per ogni struttura di controllo (`if`, `while`, `for`, `case`).
- **Automazione:** Viene utilizzato il tool `lizard`. La pipeline analizza i file di logica (JavaScript, PHP, Python) ignorando i file puramente strutturali o di stile (HTML, CSS, LaTeX). Il tool segnala la complessità media (CCN) e avvisa se specifiche funzioni superano la soglia di guardia stabilita nel Piano di Qualifica.

### 7.5.3 Visualizzazione dei risultati

L'esecuzione di questi controlli avviene automaticamente ad ogni `push` sulla repository<sub>G</sub> remota. Per consultare i risultati:

1. Accedere alla scheda **Actions** della repository<sub>G</sub> `GitHubG`.
2. Selezionare l'ultimo workflow eseguito (es. *"Compila LaTeX e Verifica<sub>G</sub> Qualità"* o *"Code Metrics"*).
3. Cliccare sui singoli job (es. *"Check Spelling"*, *"Calcolo LOC"*) per espandere il log e visualizzare i report dettagliati e le tabelle riassuntive generate dagli script.
4. Le metriche per la qualità della documentazione sono calcolate nella repository<sub>G</sub> della documentazione ufficiale del gruppo.
5. Le metriche per la qualità del codice sono calcolate nella repository<sub>G</sub> del POC e del prodotto finale.

## 7.6 Metriche nel cruscotto di qualità<sub>G</sub>

Il gruppo utilizza un cruscotto di qualità<sub>G</sub> per monitorare in modo continuo l'andamento del progetto e verificare il rispetto degli obiettivi prefissati. Il cruscotto raccoglie e aggrega un insieme di metriche significative, permettendo di ottenere una visione immediata dello stato del progetto dal punto di vista temporale, economico e qualitativo. Le principali metriche monitorate sono:

- Planned Value (PV) ed Earned Value (EV): utilizzate per confrontare il lavoro pianificato con quello effettivamente completato, consentendo di valutare l'avanzamento delle attività.
- Schedule Performance Index (SPI): permette di misurare l'efficienza temporale del progetto, evidenziando eventuali ritardi rispetto alla pianificazione.
- Cost Performance Index (CPI): consente di valutare l'efficienza economica del progetto, verificando il rapporto tra valore prodotto e costi sostenuti.
- Cost Variance (CV): indica lo scostamento tra valore prodotto e costi effettivi, permettendo di individuare eventuali inefficienze.
- Estimated at Completion (EAC): fornisce una stima aggiornata del costo finale del progetto, utile per valutare il rischio di superamento del budget.
- Requirements Stability Index (RSI): monitora la stabilità dei requisiti nel tempo, segnalando eventuali variazioni significative nello scope del progetto.
- Rischi Non Calcolati (RNC): tiene traccia dei rischi emersi durante lo sviluppo che non erano stati previsti, permettendo di valutare l'efficacia<sub>G</sub> della pianificazione iniziale.
- Metriche Soddisfatte (MS): rappresenta la percentuale di metriche che rientrano nei valori accettabili, fornendo una sintesi dello stato complessivo della qualità.

## 8 Sviluppo del POC<sub>G</sub>

Il **Proof of Concept (POC)** è stato sviluppato durante la fase RTB con l'obiettivo di validare le scelte tecnologiche e architetturali prima dello sviluppo del prodotto finale. Questa sezione definisce le norme adottate per lo sviluppo del POC, sia lato frontend che lato backend, in modo da garantire coerenza, qualità e manutenibilità del codice prodotto.

### 8.1 Stack tecnologico

Lo stack tecnologico adottato per il POC è il seguente:

- **Frontend:** React<sub>G</sub> 19 con Vite<sub>G</sub>, CSS Modules<sub>G</sub>, EasyMDE<sub>G</sub> per l'editor Markdown<sub>G</sub>;
- **Backend:** Flask<sub>G</sub> (Python), con SQLAlchemy<sub>G</sub> come ORM e psycopg2 come driver PostgreSQL<sub>G</sub>;
- **Database<sub>G</sub>:** PostgreSQL<sub>G</sub>, inizializzato tramite script SQL al primo avvio del container;
- **LLM:** API<sub>G</sub> OpenAI-compatibile fornita da Zucchetti;
- **Deployment:** Docker<sub>G</sub> e Docker Compose<sub>G</sub> per la containerizzazione dell'intera applicazione.

### 8.2 Struttura della repository<sub>G</sub> per il PoC

Il codice del POC è versionato nella repository<sub>G</sub> POC\_ByteMe. La struttura delle cartelle è la seguente:

```
POC_ByteMe/
|-- frontend/
|   |-- src/
|   |   |-- app/           # Entry point, router, provider globali
|   |   |-- components/    # Componenti riutilizzabili (Sidebar, TopBar)
|   |   |-- context/       # Context globali (Auth, History, DocName)
|   |   |-- features/      # Funzionalità per dominio applicativo
|   |       |-- auth/
|   |       |-- editor/
|   |       |-- history/
|   |       |-- personal/
|   |-- Dockerfile
|-- backend/
|   |-- src/
|   |   |-- app.py         # Entry point Flask e definizione degli endpoint
|   |   |-- auth.py        # Logica di autenticazione
|   |   |-- document.py    # Logica di gestione documenti e generazioni AI
|   |   |-- db/
|   |       |-- db.py      # Connessione SQLAlchemy e session management
|   |       |-- models.py  # Modelli ORM
|   |       |-- db.sql     # Schema del database
|-- db/
|   |-- db.sql             # Script di inizializzazione PostgreSQL
|-- docker-compose.yml
-- .env
```

## 8.3 Norme per il frontend

### 8.3.1 Architettura

Il frontend adotta la struttura **feature-based** ispirata al progetto *bulletproof-react<sub>G</sub>*. Ogni funzionalità dell'applicazione è organizzata in una cartella dedicata all'interno di **features/**, con la seguente struttura interna:

```
features/<nome-feature>/
|-- <NomeFeature>Page.jsx    # Componente pagina principale
|-- <NomeFeature>.module.css
|-- api/                     # Chiamate HTTP verso il backend
|-- hooks/                   # Custom hook con logica della feature
'-- components/              # Componenti presentazionali della feature
```

Questa struttura garantisce la *separation of concerns<sub>G</sub>*: le chiamate HTTP risiedono nel livello **api<sub>G</sub>/**, la logica applicativa nei custom hook in **hooks/**, e la presentazione nei componenti in **components/**.

### 8.3.2 Convenzioni di nomenclatura

- **Componenti React<sub>G</sub>**: PascalCase<sub>G</sub> (es. `EditorPage`, `HistoryPanel`). Questa convenzione è obbligatoria: React<sub>G</sub> distingue i componenti dai tag HTML in base alla maiuscola iniziale, e una funzione con nome minuscolo non viene trattata come componente, causando violazioni delle Rules of Hooks.
- **Custom hook**: camelCase con prefisso `use` (es. `useEditor`, `useAiHistory`).
- **File API<sub>G</sub>**: camelCase con suffisso `Call` o `ApiCall` (es. `loginCall.jsxG`, `historyApiCall.js`).
- **File CSS Modules<sub>G</sub>**: stesso nome del componente con suffisso `.module.css` (es. `EditorPage.module.css`).
- **Context**: PascalCase<sub>G</sub> con suffisso `Context` per il context, e prefisso `use` per l'hook di accesso (es. `AuthContext`, `useAuth`).

### 8.3.3 Regole sui componenti

- Ogni componente deve essere una **funzione con nome che inizia per maiuscola** ed essere usato come JSX<sub>G</sub> (`<NomeComponente />`), mai chiamato come funzione (`nomeComponente()`).
- I componenti devono essere **presentazionali**: non devono contenere logica di business o chiamate HTTP dirette. La logica va delegata ai custom hook.
- Lo stato globale condiviso tra più feature va gestito tramite **React<sub>G</sub> Context**, definito in `src/context/` e iniettato in `app/main.jsxG`.
- La navigazione tra pagine avviene tramite **React Router<sub>G</sub> v6**: per leggere lo state del router si usa l'hook `useLocation()`, mai `window.history.state` direttamente.
- Gli URL delle chiamate HTTP devono essere sempre **relativi** (es. `/apiG/login`), mai con host e porta espliciti (es. `http://localhost:8000/apiG/login`), per garantire il corretto funzionamento sia in locale che all'interno dei container Docker<sub>G</sub>.

### 8.3.4 Regole sui custom hook

- Ogni hook deve avere una responsabilità singola: gestisce lo stato e la logica di una sola operazione o funzionalità.
- Gli hook non devono essere usati condizionalmente né all'interno di cicli: le Rules of Hooks di React<sub>G</sub> devono essere sempre rispettate.
- Gli hook ricevono le dipendenze esterne (es. `getEditorText`, `nomeDocumento`) come parametri, mai lette direttamente da scope globale, per favorire la testabilità e il riuso.

### 8.3.5 Gestione dello stato

- Lo **stato locale** (es. flag di caricamento, valori di un form) viene gestito con `useState` nel componente o hook di competenza.
- Lo **stato globale** condiviso tra feature diverse (es. utente autenticato<sub>G</sub>, documento aperto, storico generazioni) viene gestito con **React<sub>G</sub> Context**, seguendo il pattern *lifting state up*.
- Non è utilizzata alcuna libreria di state management esterna (es. Redux, Zustand<sub>G</sub>): per la complessità del POC, React<sub>G</sub> Context è sufficiente.

### 8.3.6 Stile

- Lo stile è gestito con **CSS Modules<sub>G</sub>**: ogni componente ha il proprio file `.module.css`, importato come oggetto `styles`. Le classi si applicano con `className={styles.nomeClasse}`.
- Non è consentito l'uso di stile inline (`style={{ }}`) salvo casi eccezionali documentati.
- Non è consentito l'uso di classi CSS globali non scoped, per evitare conflitti tra componenti.

## 8.4 Norme per il backend

### 8.4.1 Architettura

Il backend Flask<sub>G</sub> adotta una **architettura piatta** intenzionalmente semplificata, appropriata per la natura esplorativa del POC. I file principali sono:

- `app.py`: definisce tutti gli endpoint<sub>G</sub> Flask<sub>G</sub> e gestisce le sessioni utente;
- `auth.py`: contiene la logica di registrazione e autenticazione (hashing PBKDF2);
- `document.py`: contiene la logica di gestione dei documenti, dello storico e delle generazioni AI;
- `db/db.py`: gestisce la connessione al database<sub>G</sub> tramite SQLAlchemy<sub>G</sub> e il context manager `get_db_session`;
- `db/models.py`: definisce i modelli ORM (Utente, Documento, StoricoAI, GenerazioneAI, AgenteEsterno).

#### 8.4.2 Convenzioni di nomenclatura

- **Funzioni:** snake\_case (es. `salva_nuovo_documento`, `get_generazioni_per_storico`).
- **Endpoint<sub>G</sub> Flask<sub>G</sub>:** prefisso `/apiG/` per tutte le route, con nomi descrittivi in kebab-case (es. `/apiG/save-document`, `/apiG/load-storico`).
- **Modelli ORM:** PascalCase<sub>G</sub> (es. `Utente`, `GenerazioneAI`).
- **Variabili locali:** snake\_case.

#### 8.4.3 Gestione del database<sub>G</sub>

- Tutte le operazioni sul database<sub>G</sub> avvengono tramite il context manager `get_db_session()`, che garantisce commit<sub>G</sub> automatico in caso di successo e rollback in caso di eccezione.
- Lo schema del database<sub>G</sub> è definito in `db/db.sql`. **L'ordine di definizione delle tabelle deve rispettare le dipendenze:** una tabella referenziata da una foreign key deve essere definita prima della tabella che la referencia.
- I modelli ORM in `models.py` devono essere coerenti con lo schema SQL: i nomi delle colonne e delle tabelle devono corrispondere esattamente.

#### 8.4.4 Gestione delle sessioni e sicurezza

- L'autenticazione è gestita tramite **sessioni Flask<sub>G</sub>** lato server, con cookie `HttpOnly`. La `FLASKG_SECRET_KEY` è caricata da variabile d'ambiente e non deve mai essere hardcoded nel codice.
- Le password vengono salvate nel database<sub>G</sub> esclusivamente come hash generato con **PBKDF2-HMAC-SHA256** con salt casuale, mai in chiaro.
- I messaggi di errore restituiti dagli endpoint<sub>G</sub> di autenticazione devono essere **generici**, per non rivelare informazioni sull'esistenza o meno di un utente.
- Le variabili d'ambiente sensibili (chiavi `APIG`, secret key, credenziali database<sub>G</sub>) sono definite nel file `.env` posizionato alla radice del progetto, allo stesso livello di `dockerG-compose.yml`. Il file `.env` è escluso dal versionamento<sub>G</sub> tramite `.gitignore`.

#### 8.4.5 Endpoint<sub>G</sub> REST

Ogni endpoint<sub>G</sub> deve:

- verificare la sessione utente prima di eseguire qualsiasi operazione che richieda autenticazione, restituendo 401 in caso di sessione assente o non valida;
- validare i dati in ingresso, restituendo 400 in caso di dati mancanti o malformati;
- restituire risposte in formato JSON con un campo `status` ("`success`" o "`error`") e un campo `message` descrittivo in caso di errore;
- gestire le eccezioni internamente, senza propagarle al client in forma grezza.

## 8.5 Norme per Docker<sub>G</sub> e deployment

### 8.5.1 Struttura di docker<sub>G</sub>-compose.yml

L'applicazione è composta da tre servizi:

- **db**: container PostgreSQL<sub>G</sub>. Espone la porta 5433 sull'host. Lo schema viene inizializzato automaticamente al primo avvio tramite il file montato in `/dockerG-entrypoint-initdb.d/`.
- **backend**: container Flask<sub>G</sub>. Dipende dal servizio **db** con `condition: service_healthy`, per garantire che il database<sub>G</sub> sia pronto ad accettare connessioni prima che il backend tenti di connettersi.
- **frontend**: container nginx<sub>G</sub> che serve il bundle React<sub>G</sub>. Dipende dal servizio **backend**.

### 8.5.2 Healthcheck

Il servizio **db** deve essere configurato con un healthcheck basato su `pg_isready`, in modo che Docker<sub>G</sub> attenda la disponibilità effettiva di PostgreSQL<sub>G</sub> prima di avviare il backend. Il mancato utilizzo dell'healthcheck causa errori di connessione al database<sub>G</sub> al primo avvio.

### 8.5.3 Variabili d'ambiente

Le variabili d'ambiente sono passate al container **backend** tramite il file `.env` letto da Docker Compose<sub>G</sub>. Le variabili necessarie sono:

- `OPENAI_API_KEY`: chiave per l'API<sub>G</sub> LLM di Zucchetti;
- `OPENAI_BASE_URL`: URL base dell'API<sub>G</sub> LLM;
- `FLASKG_SECRET_KEY`: chiave segreta per le sessioni Flask<sub>G</sub>.

### 8.5.4 Comandi

- **Primo avvio o dopo modifiche**: `docker composeG down -v && docker composeG up --build`. Il flag `-v` elimina i volumi e forza la reinizializzazione del database<sub>G</sub>.
- **Avvio normale**: `docker composeG up --build`.
- **Arresto**: `docker composeG down`.

## 9 Processo di sviluppo del Minimum Viable Product

### 9.1 Attività: sviluppo del Minimum Viable Product

Questa attività riguarda la realizzazione tecnica del prodotto *Second Brain*, evolvendo il prototipo iniziale in un sistema modulare, scalabile e containerizzato.

### 9.2 Procedura: Implementazione dell'architettura frontend (Bulletproof React<sub>GG</sub>)

Ogni membro del team addetto al frontend deve seguire la struttura *feature-based* per garantire l'isolamento delle funzionalità.

1. **Scomposizione in Feature:** Ogni nuova funzionalità deve essere isolata nella directory `src/features/`.
2. **Standardizzazione dei Sottomoduli:** Ogni cartella di feature deve obbligatoriamente contenere le directory `apiG/`, `components/`, `hooks/`, `store/` e `types/`.
3. **Incapsulamento (Interfaccia Pubblica):** L'accesso alle risorse interne di una feature è consentito esclusivamente tramite il file `index.ts`. È vietato l'import diretto di file interni da parte di moduli esterni (Basso Accoppiamento).
4. **Gestione dello Stato:** Per la persistenza locale e la gestione dei metadati deve essere utilizzato lo store *Zustand<sub>G</sub>* con middleware di persistenza su *LocalStorage*.

### 9.3 Procedura: sviluppo del backend stateless<sub>G</sub>

Il backend deve agire esclusivamente come gateway di comunicazione.

1. **Assenza di Stato:** È vietata l'implementazione di `databaseG` persistenti lato server. Ogni richiesta deve essere autocontenuta. `operation_registry.py` per disaccoppiare il controller dalla logica delle dipendenze.
2. **Strategie di prompt<sub>G</sub>:** L'elaborazione dei testi deve seguire il pattern *Strategy*, isolando la costruzione dei `promptG` in classi specifiche in `ai_strategies.py`.

### 9.4 Strumenti e organizzazione repository<sub>G</sub>

L'ambiente di sviluppo e la struttura del codice sorgente sono così organizzati:

- **Linguaggi e Framework:** React<sub>G</sub> 19, TypeScript, Python 3.12 (Flask<sub>G</sub>).
- **Infrastruttura:** Docker<sub>G</sub> e Docker Compose<sub>G</sub> per la gestione dei container.

```
PB_ByteMe/
|-- docker-compose.yml
|-- backend/
|   |-- Dockerfile
|   |-- requirements.txt
|   |-- src/
|   |   |-- app.py                # Entry point Flask (Routing)
|   |   |-- operation_registry.py # Registry pattern per le operazioni AI
|   |   |-- ai_strategies.py      # Strategie di prompt (es. Summary, De Bono)
|   |   |-- model_strategies.py   # Logica di comunicazione con API
|   |   +-- utils/
|   +-- test/                    # Suite di test unitari (Pytest)
```



```

|
+-- frontend/
  |-- Dockerfile
  |-- package.json
  |-- vite.config.js
  +-- src/
    |-- app/                # Configurazione globale e routing
    |-- assets/
    |-- components/         # Componenti UI globali
    |-- lib/                # Configurazioni librerie esterne
    |-- styles/             # CSS globali
    |-- types/              # Tipi globali
    +-- features/           # Moduli Bulletproof React
      |-- editor/           # Logica e UI dell'editor Markdown
      |   |-- api/
      |   |-- components/
      |   |-- hooks/
      |   |-- store/        # Gestione stato editor (Zustand)
      |   |-- utils/
      |   +-- index.ts      # API pubblica della feature editor
      +-- history/         # Logica e UI dello storico AI
          |-- components/
          |-- hooks/
          |-- store/        # Persistenza storico (Zustand persist)
          |-- types/
          +-- index.ts      # API pubblica della feature history

```

## 10 Processo di Verifica<sub>G</sub>

### 10.1 Attività: testing e validazione<sub>G</sub> continua

L'attività assicura che ogni incremento di codice rispetti i requisiti qualitativi e funzionali prefissati.

### 10.2 Procedura: Esecuzione test unitari

Tutto il codice prodotto deve essere coperto da test automatici per prevenire regressioni.

- **Frontend:** Utilizzo di *Vitest<sub>G</sub>* e *React<sub>G</sub> Testing Library*. I file di test devono risiedere nella stessa cartella del componente o dell'hook verificato.
- **Backend:** Utilizzo di *Pytest<sub>G</sub>* per la verifica<sub>G</sub> del routing e delle logiche di pulizia testo.

### 10.3 Procedura: automazione quality gate (Pre-push)

Per garantire che la repository<sub>G</sub> remota contenga solo codice stabile, viene imposto l'uso di *Git Hooks<sub>G</sub>*.

1. **Trigger:** Al comando `gitG push`, il sistema invoca automaticamente gli script di controllo.
2. **Validazione<sub>G</sub>:** Vengono eseguiti in sequenza `run_tests.sh` (Backend) e `test_frontend.sh` (Frontend).
3. **Blocco:** Se l'exit code di uno degli script è diverso da zero, l'operazione di caricamento viene interrotta forzatamente.

## 11 Processo di supporto (Qualità)

### 11.1 Attività: valutazione e selezione dei LLM

Questa attività definisce il protocollo per la scelta del modello linguistico più idoneo a ogni funzionalità AI. Il gruppo adotta un protocollo rigoroso per la selezione degli LLM (*Large Language Model*) da integrare nell'MVP, al fine di bilanciare capacità creative e rigore analitico.

### 11.2 Procedura: protocollo di testing comparativo e metriche

Ogni qualvolta sia necessario assegnare un modello a una specifica funzionalità (es. Traduzione, Analisi Critica), i membri del team devono seguire la seguente procedura:

1. **Campionamento:** Selezione di un set di dati sorgente (*ground truth<sub>G</sub>*) rappresentativo del task.
2. **Configurazione Parametrica:** Fissazione della temperatura<sub>G</sub> (es. 0.1 per task deterministici, maggiore di 0.7 per task creativi) per garantire la riproducibilità.
3. **Esecuzione e reportistica:** Ogni membro esegue test individuali su diversi modelli (es. Llama, Gemma, Qwen) e redige un resoconto tecnico.

La scelta del modello finale è vincolata al superamento delle seguenti soglie quantitative, misurate manualmente o automaticamente:

- **Aderenza al ruolo (AR):** Valutazione manuale su scala discreta (0, 1, 2). Un modello è accettabile se ottiene un punteggio = 2 nell'80% dei campioni.
- **Conformità strutturale (CS):** Verifica<sub>G</sub> binaria del rispetto dei vincoli di formato (es. output solo in Markdown<sub>G</sub>). Soglia minima: 80%.
- **Tasso di allucinazione (HR):** Controllo manuale della fedeltà al testo sorgente. Non deve superare il 5%.
- **Efficienza temporale:** Misurazione automatica della latenza. Il tempo di risposta deve essere  $\leq 60$  secondi per sessione.

*Nota sull'efficienza temporale:* Si precisa che la latenza registrata non è un valore strettamente dipendente dalla sola complessità computazionale (numero di parametri) del modello scelto. Come indicato dal proponente Zucchetti, il tempo di risposta totale è influenzato in modo significativo da fattori infrastrutturali esterni, quali il carico istantaneo di lavoro sui server condivisi e i tempi tecnici di allocazione delle risorse hardware. Nello specifico, il processo di "attivazione" del modello sulla macchina di esecuzione può generare overhead variabili, rendendo la latenza un parametro non deterministico: un modello nominalmente più leggero potrebbe comunque presentare tempi di risposta superiori in scenari di congestione o in caso di inizializzazione a freddo (*cold start*).